Master Prompt — Production-Scale Vacation Rental Marketplace Architecture

Built for the Playpower Labs "Airbnb-Clone" take-home assignment. Paste the block below into Claude, ChatGPT, or any AI/diagramming assistant to generate the strongest possible architecture diagram + write-up for your submission.

Why this prompt is built this way

The assignment asks for **one deliverable most candidates under-invest in**: a high-level architecture diagram for a *production-scale* version of the app, covering scaling strategy for **frontend, backend, storage, search, and deployment**. Reviewers explicitly grade "production architecture thinking" as a separate axis from UI fidelity. This prompt is written so the output:

1. Goes far beyond "React app → Node API → Postgres" (the naive answer most submissions give).
2. Explicitly names real, defensible technology choices with a one-line justification each (so you can speak to every box in an interview).
3. Is structured as a **C4-style, layered diagram** (Context → Container → key data flows) so it's readable to both engineers and non-technical reviewers.
4. Mirrors the grading rubric almost line-for-line, so nothing in "what we will look at" is left uncovered.

The Prompt (copy everything between the lines)

```
You are a Principal Solutions Architect at a company operating a global,
production-grade vacation-rental marketplace (Airbnb-scale): tens of
millions of listings, hundreds of millions of monthly users, multi-region
traffic, real-money payments, and a search experience that must return
geo-ranked, filtered results in under 200ms at p95.

Design and diagram the system's production architecture. This is NOT the
take-home clone itself — it is the "how would this scale to real-world
production" architecture that accompanies it. Treat the take-home app
(a listing page + photo tour + lightbox, built in React/Next.js, optionally
with a lightweight backend or browser storage) as the seed of the frontend
client only. Everything else must be designed as if this were a real,
```

venture-scale company.

Produce two things:

═══

PART 1 — ARCHITECTURE DIAGRAM (structure to render)

═══

Use a C4-style layered layout with swimlanes, left to right or top to bottom, and clear arrows for data/request flow. Include these layers, and inside each layer name specific real technologies (not "a database" — say "Postgres, sharded by geo-region, with read replicas per region"):

1. EDGE / CLIENT LAYER
   - Global CDN + edge cache (Cloudflare / CloudFront / Fastly) serving static assets, hashed bundles, and cached HTML for anonymous traffic
   - Next.js frontend using SSR for SEO-critical listing pages, ISR (incremental static regeneration) for listings that change infrequently, and CSR for logged-in/interactive states (booking flow, wishlist, host dashboard)
   - Image pipeline: on-the-fly resizing/format negotiation (AVIF/WebP) via an image CDN (Cloudflare Images / Imgix / Thumbor), srcset + lazy-loading + blur-up placeholders for the photo tour & lightbox
   - Client-side performance budget: code-splitting per route, prefetch on hover/viewport-intersection for listing cards, Core Web Vitals (LCP/INP/CLS) as an explicit NFR
   - A/B testing & feature flag service (LaunchDarkly/Optimizely-style) sitting between edge and origin

2. API / GATEWAY LAYER
   - API Gateway (Kong / AWS API Gateway / Envoy) handling authn/z, rate limiting, request routing, and TLS termination
   - Backend-for-Frontend (BFF) per client surface (web BFF, host-app BFF) aggregating calls to downstream services so the client makes one round trip instead of many
   - GraphQL federation OR REST + OpenAPI — pick one and justify it (GraphQL federation reduces over-fetching for the highly nested listing page: host, amenities, reviews, availability, pricing)
   - WAF + bot protection + DDoS mitigation at this layer

3. CORE DOMAIN SERVICES (microservices, independently deployable/scalable)
   - Identity & Access (OAuth2/OIDC, JWT, session mgmt, social login)
   - Listings Service (CRUD, ownership, amenities, house rules)
   - Search & Discovery Service (query parsing, ranking, personalization)
   - Availability & Calendar Service (high write-throughput, per-listing

date-range locking to prevent double-booking)
   - Pricing Service (dynamic/smart pricing, currency conversion)
   - Booking & Reservation Service (orchestrates the booking saga)
   - Payments Service (PCI-scope isolated; wraps Stripe/Adyen; never touches raw card data — tokenization only)
   - Messaging/Inbox Service (guest↔host chat, near-real-time)
   - Reviews & Ratings Service
   - Trust & Safety / Fraud Detection Service (risk scoring on signup, listing creation, and booking)
   - Notifications Service (email/SMS/push fan-out)
   - Media Service (upload, transcode, virus/NSFW scan, store to object storage, emit CDN-ready URLs)
   - Host Tools / Analytics Service (dashboards, payouts, calendar sync with iCal/other OTAs)

   Each service: independently deployable, owns its own datastore (database-per-service), communicates via async events for cross-service side effects and sync gRPC/REST for request/response.

4. DATA LAYER (polyglot persistence — justify each choice)
   - Transactional core (Users, Listings, Bookings, Payments): PostgreSQL, horizontally sharded by geo-region/listing-id, with read replicas per region and a connection pooler (PgBouncer)
   - Caching: Redis cluster for session data, hot listing pages, rate-limit counters, and availability-calendar read-through cache
   - Search index: Elasticsearch/OpenSearch cluster, geo-spatial (geo_point/geo_shape) indexing for map search, denormalized listing documents rebuilt via CDC (Debezium) from Postgres
   - High-write / flexible schema data (chat messages, activity logs, availability events): Cassandra or DynamoDB
   - Object storage: S3 (or GCS) for photos/media, fronted by the CDN
   - Analytics / data warehouse: events streamed to Snowflake/BigQuery for BI, pricing models, and search-ranking model training
   - Change Data Capture (CDC) pipeline (Debezium + Kafka) keeping Elasticsearch, caches, and the data warehouse in sync with the source-of-truth Postgres instances

5. SEARCH & RANKING (call this out as its own critical path, since the assignment names it explicitly)
   - Query layer: parses free-text + structured filters (dates, guests, price, amenities) into an Elasticsearch bool/geo query
   - Geo-search: geohash/geo_point queries for map-bounds search, pre-aggregated tile-based clustering for pins at low zoom levels
   - Ranking: base relevance (ES score) blended with a learned ranking model (quality score, conversion likelihood, host responsiveness),

served via a ranking/re-ranking microservice
   - Personalization: user embeddings / recently-viewed signals folded in at re-ranking time, not at the base query, to keep the ES query cacheable
   - Caching: popular/no-filter searches cached at the edge and in Redis with short TTL + cache-busting on price/availability change

6. ASYNC / EVENT-DRIVEN BACKBONE
   - Kafka (or Kinesis/Pub-Sub) as the central event bus
   - Booking flow modeled as a SAGA (reserve-hold → payment-authorize → confirm → notify) with compensating transactions on failure (e.g., release the date-hold if payment fails)
   - Outbox pattern on each service's DB to guarantee at-least-once event publication without dual-write inconsistency
   - Event consumers: search-index updater, notification dispatcher, analytics pipeline, fraud-scoring pipeline

7. DEPLOYMENT & INFRASTRUCTURE
   - Kubernetes (EKS/GKE/AKS), one cluster per region, services as independently scaled Deployments with HPA (CPU/RPS-based) and KEDA for event-driven scaling (e.g., media transcoding queue depth)
   - Service mesh (Istio/Linkerd) for mTLS between services, retries, circuit breaking, and traffic shifting (canary/blue-green)
   - Multi-region active-active for read-heavy services (search, listings) and active-passive with failover for the payments/booking path (to keep transactional consistency simpler)
   - Global traffic management: DNS-based geo-routing (Route 53 / Cloudflare Load Balancer) sending users to nearest healthy region
   - IaC: Terraform for all cloud resources, GitOps (ArgoCD/Flux) for Kubernetes manifests
   - CI/CD: GitHub Actions/GitLab CI → build → test → containerize → canary deploy → automated rollback on SLO breach
   - Disaster recovery: defined RPO/RTO targets, cross-region backups, regular game-day/chaos testing (Chaos Mesh / Gremlin)

8. OBSERVABILITY & OPERATIONS
   - Metrics: Prometheus + Grafana, SLO dashboards per service
   - Distributed tracing: OpenTelemetry → Jaeger/Tempo, so a single booking request can be traced across all 6+ services it touches
   - Centralized logging: ELK/Loki
   - Alerting/on-call: PagerDuty/Opsgenie tied to SLO burn-rate alerts
   - Synthetic monitoring on the critical path (search → listing → booking → payment)

9. SECURITY & COMPLIANCE
   - PCI-DSS scope isolated to the Payments service (tokenized cards only, never stored elsewhere)
   - Encryption in transit (mTLS everywhere) and at rest (KMS-managed keys per datastore)
   - Secrets management (Vault / AWS Secrets Manager)
   - GDPR/data residency: EU user data pinned to EU region shards
   - Rate limiting + WAF + bot detection at the edge (covered above, reference again here as defense-in-depth)

Render this as a single diagram with 9 clearly labeled zones/swimlanes matching the sections above, arrows showing the request path for the three most important flows (search → listing page load, browse-to-book saga, photo upload → CDN-served image), and a small legend distinguishing sync (solid arrow) vs async/event (dashed arrow) communication.

═══

PART 2 — ONE-PAGE WRITTEN RATIONALE (accompany the diagram)

═══

For each of the 5 areas the assignment explicitly grades — Frontend scaling, Backend scaling, Storage scaling, Search scaling, Deployment scaling — write 3-4 sentences covering:
   a) The bottleneck this app would hit first at 10x, then 100x scale
   b) The specific mechanism from the diagram that addresses it
   c) One explicit trade-off you accepted (e.g., "active-passive payments sacrifices some multi-region latency for simpler consistency")

Close with a short "what I'd build first vs. defer" section — a production system isn't built all-at-once; name the 3 components you'd stand up in month 1 (monolith-first is a valid, defensible answer here) versus the 3 you'd only introduce once traffic justifies the complexity (e.g., don't shard Postgres or stand up Kafka on day one — name the metric/threshold that would trigger that migration).

Tone: precise, decisive, and free of hedging. Prefer specific numbers (p95 latency targets, RPS, data volume) over vague adjectives like "scalable" or "robust." Do not pad with generic cloud-architecture boilerplate that isn't tied to this specific product (a two-sided vacation-rental marketplace with search, real-money payments, and media-heavy listings).

How to actually turn this into the submitted diagram

The assignment recommends **Excalidraw** or **Lucidchart**. A workflow that works well:

1. Paste the prompt above into Claude (or another model) and ask it to output the diagram as **Mermaid** or **structured text** (boxes/arrows/labels) rather than prose.
2. Paste that structure into Excalidraw (there's a "Mermaid to Excalidraw" import) or manually lay it out in Lucidchart using the 9 swimlanes as columns/rows.
3. Export as PNG/PDF — that's your `architecture-diagram` deliverable.
4. Keep the Part 2 write-up as a short accompanying doc or as annotations/notes directly on the diagram — reviewers are explicitly told to look for "production architecture thinking," so a diagram with zero explanatory text underused this part of the grading.

Quick self-check against the grading rubric before you submit

- [ ] Frontend scaling strategy is explicit (CDN, SSR/ISR, image pipeline) — not just "React app"
- [ ] Backend scaling strategy is explicit (microservices, independent scaling, service mesh) — not just "Node API"
- [ ] Storage scaling strategy is explicit (sharding, replicas, polyglot persistence, CDC) — not just "Postgres"
- [ ] Search is called out as its own component (Elasticsearch/OpenSearch + geo + ranking) — the assignment names this separately from "storage," don't merge them
- [ ] Deployment strategy is explicit (multi-region, K8s, CI/CD, IaC) — not just "deploy to Vercel"
- [ ] The diagram is legible at a glance — a non-engineer reviewer should be able to follow the search→listing→booking flow just from the arrows
- [ ] You can defend every box in a follow-up conversation — don't include a technology you can't explain the trade-offs of